

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)

Институт
информационных технологий

Кафедра
управления и информатики
в технических системах

ОТЧЕТ О ВЫПОЛНЕНИИ

ЛАБОРАТОРНОЙ РАБОТЫ ПО ДИСЦИПЛИНЕ

«Прикладное программирование»

по теме «ООП в Python. Обработка исключительных ситуаций. Форматы XML и JSON»

СТУДЕНТА *2* КУРСА бакалавриата ГРУППЫ ИДБ-25-12

(уровень профессионального образования)

Баязитова Ильсина Юлдыбаевна

Направление: 09.03.03 Прикладная информатика

Отчет сдан «_____» _____2026г.

Оценка _____

Преподаватель Гетманов С. В. _____

(Ф.И.О., должность, степень, звание.) *(подпись)*

МОСКВА 2026

Оглавление

1. Цель работы 3

2. Постановка задачи 3

3. Описание предметной области 4

4. Проектирование классов UML 5

5. Реализация программы 7

5.1. Консольный интерфейс 8

6. Обработка исключительных ситуаций 9

7. Хранение данных в JSON и XML 10

8. Проверка работы программы 12

9. Заключение 13

1. Цель работы

Цель лабораторной работы – закрепить навыки объектно-ориентированного программирования на Python, научиться проектировать диаграмму классов, создавать и обрабатывать встроенные и собственные исключения, а также реализовывать чтение и запись структурированных данных в форматах JSON и XML.

2. Постановка задачи

Для предметной области «Прокат велосипедов» необходимо выполнить следующие задачи:

- Спроектировать в сервисе draw.io диаграмму классов для выбранной предметной области и написать по ней код на Python в объектно-ориентированном стиле.
- Обработать встроенные и собственные исключения, чтобы ошибки пользователя не завершали программу; после чего актуализировать диаграмму классов.
- Спроектировать в текстовом редакторе структуру для хранения данных предметной области в форматах XML и JSON.
- Написать код на Python, реализующий считывание из файла и запись в файл по разработанной структуре в форматах XML и JSON.

3. Описание предметной области

Выбрана собственная предметная область «Прокат велосипедов». Сеть станций проката выдаёт клиентам городские велосипеды и электровелосипеды с аксессуарами (шлем, замок, детское кресло). Клиент арендует велосипед на временной интервал, стоимость считается по цене за час. Аренда невозможна, если велосипед уже арендован на это время, клиент младше допустимого возраста (12 лет для городского, 16 для электровелосипеда) или аренда длиннее допустимого (до 24 часов для городского, до 8 для электровелосипеда). Аренда электровелосипеда требует подтверждения залога.

4. Проектирование классов UML

Система описана 17 классами (рис. 1). Абстрактные классы выделены курсивом. Диаграмма построена в draw.io.

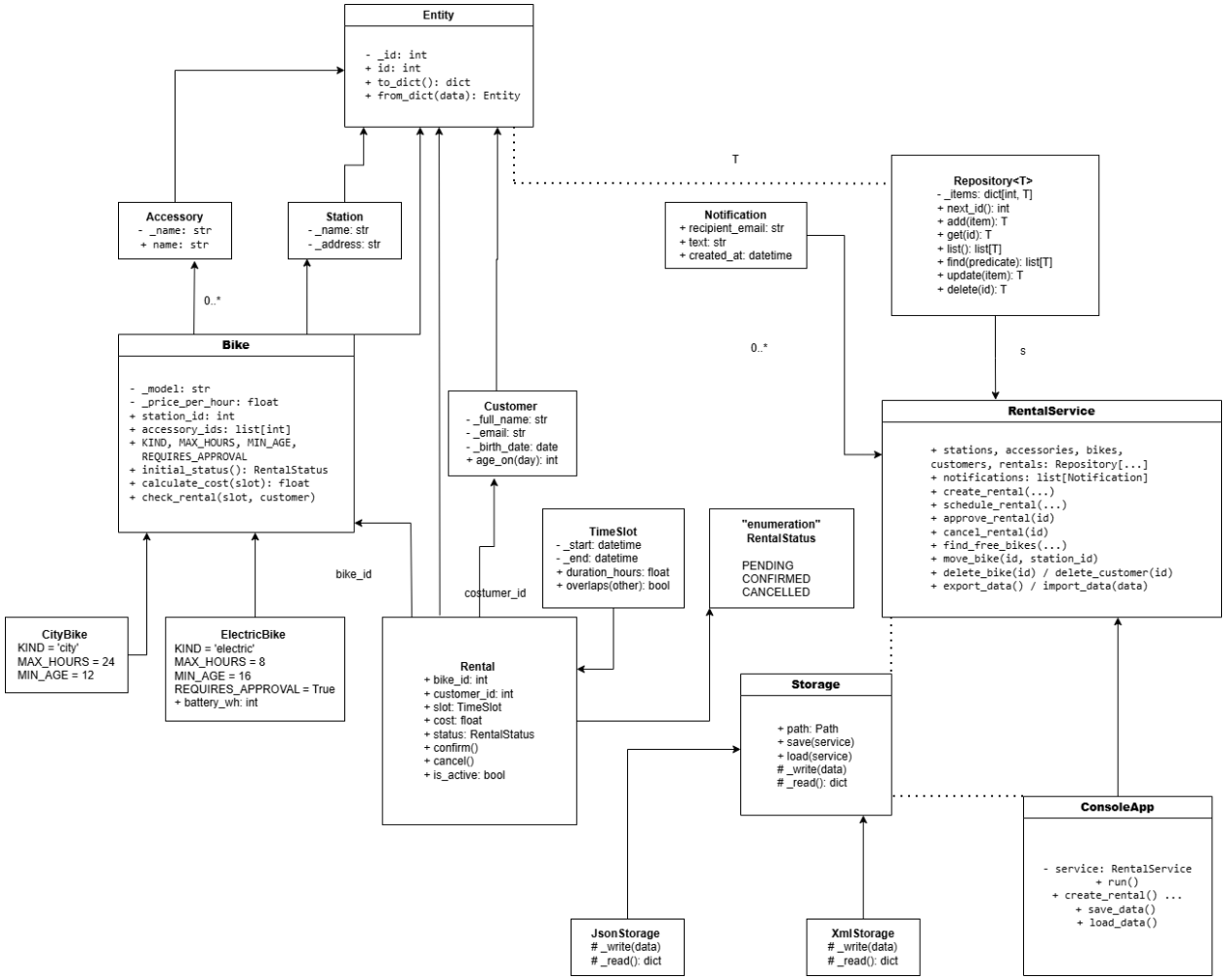


Рис. 1. Диаграмма классов (до добавления исключений)

Таблица 1. Назначение классов

Класс	Назначение
Entity	абстрактный базовый класс всех хранимых объектов (id, to_dict, from_dict)
Accessory, Station	аксессуар и станция проката
Customer	клиент (ФИО, email, дата рождения, возраст на дату)
Bike	абстрактный велосипед: цена, станция, аксессуары, проверка аренды
CityBike, ElectricBike	городской и электровелосипед (разные лимиты времени и возраста, залог)
Rental	аренда: велосипед, клиент, интервал, стоимость, статус
TimeSlot	интервал времени, проверка пересечения
RentalStatus	перечисление статусов аренды
Notification	уведомление клиенту
Repository[T]	обобщённое хранилище с операциями CRUD
RentalService	бизнес-логика проката, поиск свободных велосипедов
Storage, JsonStorage, XmlStorage	запись и чтение файлов JSON и XML
ConsoleApp	консольное меню

5. Реализация программы

Программа написана на языке Python в объектно-ориентированном стиле. Бизнес-логика проката (оформление аренды с проверкой занятости велосипеда, возраста клиента и допустимой длительности, подтверждение залога, поиск свободных велосипедов) сосредоточена в классе RentalService. Объекты хранятся в обобщённом хранилище Repository[T] с операциями CRUD.

Запись и чтение данных выполняют классы JsonStorage и XmlStorage на основе модулей json и xml.etree.ElementTree. Взаимодействие с пользователем организовано через консольное меню (класс ConsoleApp), графический интерфейс не используется. Сценарий demo.py демонстрирует штатную работу программы и все виды обрабатываемых ошибок.

5.1. Консольный интерфейс

Сеанс в консольном меню: неверный ввод, конфликт по времени, возрастное ограничение, неверный формат даты, удаление арендованного велосипеда. После каждой ошибки программа возвращается в меню.

```
Выберите пункт: abc
Нет такого пункта меню

Выберите пункт: 3
id велосипеда: x
Некорректный ввод: «x» - не целое число

Выберите пункт: 3
id велосипеда: 1
id клиента: 1
Начало (ГГГГ-ММ-ДДТЧЧ:ММ): 2026-11-10T10:00
Окончание (ГГГГ-ММ-ДДТЧЧ:ММ): 2026-11-10T13:00
Оформлено: №1, 10.11.2026 10:00-10.11 13:00, 450 руб., подтверждена

Выберите пункт: 3
id велосипеда: 1
id клиента: 3
Начало (ГГГГ-ММ-ДДТЧЧ:ММ): 2026-11-10T12:00
Окончание (ГГГГ-ММ-ДДТЧЧ:ММ): 2026-11-10T14:00
Ошибка: Велосипед «Stels Navigator» уже арендован (аренда №1)

Выберите пункт: 3
id велосипеда: 3
id клиента: 2
Начало (ГГГГ-ММ-ДДТЧЧ:ММ): 2026-11-11T10:00
Окончание (ГГГГ-ММ-ДДТЧЧ:ММ): 2026-11-11T12:00
Ошибка: Велосипед «Xiaomi Mi E-Bike» выдаётся с 16 лет, а клиенту 14

Выберите пункт: 3
id велосипеда: 1
id клиента: 1
Начало (ГГГГ-ММ-ДДТЧЧ:ММ): 10.11.2026
Ошибка: Дата «10.11.2026» не соответствует формату ГГГГ-ММ-ДДТЧЧ:ММ

Выберите пункт: 11
id велосипеда: 1
Ошибка: Нельзя удалить велосипед с активными арендами

Выберите пункт: 2
№1, 10.11.2026 10:00-10.11 13:00, 450 руб., подтверждена | Stels Navigator | Иванова Мария Петровна

Выберите пункт: 0
До свидания!
```

6. Обработка исключительных ситуаций

Созданы собственные исключения с общим предком BikeRentalError (табл. 2). Главный цикл меню перехватывает BikeRentalError, ValueError (неверный ввод числа или даты), EOFError и KeyboardInterrupt, а также любое непредвиденное Exception, поэтому ошибка не завершает программу.

Таблица 2. Собственные исключения

Исключение	Когда возникает
BikeRentalError	базовый класс ошибок приложения
ValidationError	пустое имя, неверный email или дата, конец аренды раньше начала, слишком длинная аренда
EntityNotFoundError	объект с таким id не найден
DuplicateEntityError	объект с таким id уже есть
DependencyError	удаление объекта, на который есть ссылки
RentalConflictError	велосипед уже арендован на это время
AgeRestrictionError	клиент младше минимального возраста
InvalidStatusError	недопустимая смена статуса аренды
StorageError	ошибка чтения или записи JSON/XML

Встроенные исключения обрабатываются так: ValueError – при вводе чисел и дат; KeyError, TypeError, AttributeError – при файле неверной структуры; FileNotFoundError и OSError – при недоступном файле; json.JSONDecodeError и xml.etree.ElementTree.ParseError – при повреждённом JSON или XML. Файловые ошибки преобразуются в StorageError с понятным сообщением.

После добавления исключений диаграмма актуализирована (рис. 2).

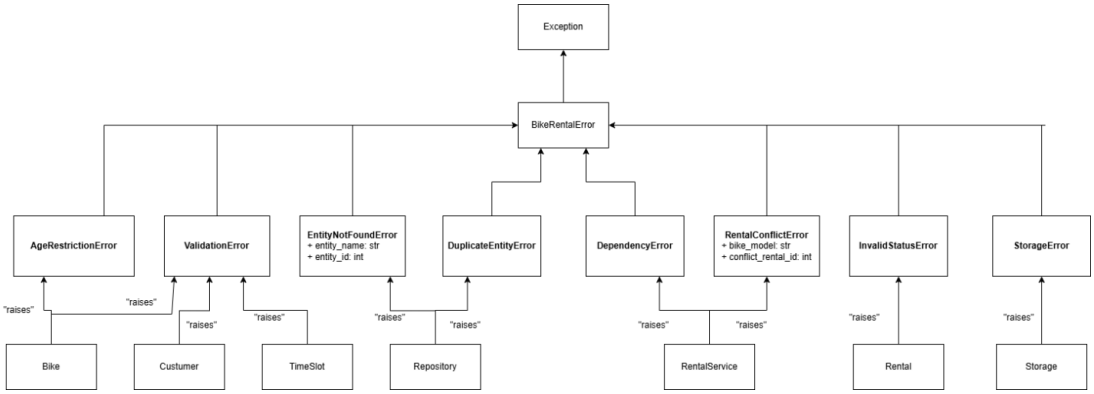


Рис. 2. Актуализированная диаграмма: иерархия исключений и места их возникновения

7. Хранение данных в JSON и XML

В обоих форматах данные разбиты на пять коллекций: stations, accessories, bikes, customers, rentals. Связи между объектами заданы числовыми id. Тип велосипеда хранится в поле kind. В XML списки помечены атрибутом list="true".

Фрагмент JSON (data/data.json):

```
{
  "bikes": [
    {
      "id": 3,
      "kind": "electric",
      "model": "Xiaomi Mi E-Bike",
      "station_id": 1,
      "price_per_hour": 400,
      "accessory_ids": [
        1,
        2
      ],
      "battery_wh": 480
    }
  ],
  "customers": [
    {
      "id": 1,
      "full_name": "Иванова Мария Петровна",
      "email": "ivanova@example.com",
      "birth_date": "2000-03-14"
    }
  ],
  "rentals": [
    {
      "id": 1,
      "bike_id": 1,
      "customer_id": 1,
      "start": "2026-11-04T10:00",
      "end": "2026-11-04T12:00",
      "cost": 300,
      "status": "confirmed"
    }
  ]
}
```

Фрагмент XML (data/data.xml):

```
<bike_rental>
  <bikes>
    <bike>
      <id>3</id>
      <kind>electric</kind>
      <model>Xiaomi Mi E-Bike</model>
      <station_id>1</station_id>
      <price_per_hour>400.0</price_per_hour>
      <accessory_ids list="true">
        <item>1</item>
        <item>2</item>
      </accessory_ids>
      <battery_wh>480</battery_wh>
    </bike>
    ...
  </bikes>
  <rentals>
    <rental>
      <id>1</id>
      <bike_id>1</bike_id>
      <customer_id>1</customer_id>
      <start>2026-11-04T10:00</start>
      <end>2026-11-04T12:00</end>
      <cost>300.0</cost>
      <status>confirmed</status>
    </rental>
    ...
  </rentals>
</bike_rental>
```

Запись и чтение реализованы классами JsonStorage (модуль json) и XmlStorage (модуль xml.etree.ElementTree). Проверено, что после записи и загрузки данные совпадают с исходными.

8. Проверка работы программы

Сценарий demo.py: обычная работа и все виды ошибок (ожидаемые ошибки обработаны, программа дошла до конца).

Таблица 3. Результаты проверки

Проверка	Ожидаемый результат
Штатная работа	Оформление аренды городского велосипеда и электровелосипеда (в том числе «встык»), подтверждение залога, перенос аренды, перевод велосипеда на другую станцию и поиск свободных велосипедов выполняются без ошибок.
Конфликт по времени	Аренда уже занятого на это время велосипеда отклоняется с RentalConflictError.
Возрастное ограничение	Клиент младше минимального возраста получает AgeRestrictionError.
Проверка интервала аренды	Конец аренды раньше начала и аренда длиннее допустимой отклоняются с ValidationError.
Проверка полей	Некорректный email и неверный формат даты рождения отклоняются с ValidationError.
Поиск объектов	Обращение к несуществующему велосипеду вызывает EntityNotFoundError.
Смена статуса	Повторное подтверждение аренды вызывает InvalidStatusError.
Проверка зависимостей	Удаление арендованного велосипеда и станции, на которой остались велосипеды, отклоняется с DependencyError.
JSON	Данные записываются классом JsonStorage и читаются обратно; результат совпадает с исходными данными.
XML	Данные записываются классом XmlStorage и читаются обратно; результат совпадает с исходными данными.
Файловые ошибки	Некорректный JSON или XML, файл неверной структуры и отсутствующий файл приводят к StorageError с пояснением причины.

Полный вывод сценария demo.py:

```
--- Нормальная работа ---
[OK]     аренда городского: №1, 02.11.2026 10:00-02.11 13:00, 450 руб., подтверждена
[OK]     аренда «встык»: №2, 02.11.2026 13:00-02.11 15:00, 300 руб., подтверждена
[OK]     аренда электро: №3, 03.11.2026 09:00-03.11 12:00, 1200 руб., ожидает залога
[OK]     подтверждение залога: №3, 03.11.2026 09:00-03.11 12:00, 1200 руб., подтверждена
[OK]     перенос аренды: №1, 04.11.2026 10:00-04.11 12:00, 300 руб., подтверждена
[OK]     перевод на другую станцию: Forward Racer (120 руб./ч)
[OK]     свободные велосипеды: ['Stels Navigator']

--- Обработка ошибок ---
[ОШИБКА] занят (своё исключение): RentalConflictError: Велосипед «Stels Navigator» уже арендован (аренда №1)
[ОШИБКА] клиент слишком молод: AgeRestrictionError: Велосипед «Xiaomi Mi E-Bike» выдаётся с 16 лет, а клиенту
14
[ОШИБКА] конец раньше начала: ValidationError: Время окончания должно быть позже начала
[ОШИБКА] слишком долго: ValidationError: Аренда «Xiaomi Mi E-Bike» не может быть длиннее 8 ч.
[ОШИБКА] нет такого велосипеда: EntityNotFoundError: Велосипед с id=99 не найден(а)
[ОШИБКА] повторное подтверждение: InvalidStatusError: Подтвердить можно только аренду в ожидании залога, а
сейчас: подтверждена
[ОШИБКА] удаление арендованного: DependencyError: Нельзя удалить велосипед с активными арендами
[ОШИБКА] удаление станции с велосипедами: DependencyError: На станции остались велосипеды
[ОШИБКА] некорректный email: ValidationError: Некорректный email: без-собаки
[ОШИБКА] неверная дата рождения: ValidationError: Дата «01.01.1990» не соответствует формату ГГГГ-ММ-ДД

--- JSON и XML ---
JsonStorage: записано, прочитано, данные совпадают = True
XmlStorage: записано, прочитано, данные совпадают = True
```

```
--- Повреждённые файлы ---
[ОШИБКА] загрузка bad.json: StorageError: Файл bad.json - не корректный JSON: Expecting property name enclosed
in double quotes: line 1 column 3 (char 2)
[ОШИБКА] загрузка bad.xml: StorageError: Файл bad.xml - не корректный XML: no element found: line 1, column 20
[ОШИБКА] загрузка wrong.json: StorageError: Файл wrong.json имеет неверную структуру (ValueError: invalid
literal for int() with base 10: 'x')
[ОШИБКА] файла нет: StorageError: Файл data/none.json не найден
```

9. Заключение

В ходе лабораторной работы для предметной области «Прокат велосипедов» в сервисе draw.io спроектирована диаграмма из 17 классов и написана программа на Python в объектно-ориентированном стиле. Разработана иерархия собственных исключений с общим предком BikeRentalError; встроенные и собственные исключения обрабатываются так, что ошибки пользователя не завершают программу, а диаграмма классов актуализирована.

Разработана структура хранения данных в форматах JSON и XML, реализованы чтение и запись файлов классами JsonStorage и XmlStorage; проверено, что после записи и загрузки данные совпадают с исходными. Поставленные задачи лабораторной работы выполнены.

Ссылка на репозиторий GitHub: https://github.com/nexus-100/labs